EAST WEST UNIVERSITY

# Implementation of Dynamic Table

CSE207 Data Structure

Mini Project

Submitted by-


| Name | ID |
|---|---|
| **Rifa Azad** | 2022-1-60-265 |
| **Tasnuva Islam Mila** | 2022-1-60-033 |

**Submitted To –**

**Dr. Maheen Islam**

Chairperson, Associate Professor

Department of Computer Science & Engineering

# Table of Contents


| Index | Page |
|---|---|
| 1) Introduction ---------- | 3 |
| 2) Methodology ---------- | 4 |
| a) Approach | |
| b) Process | |
| c) Detail Explanation of each Function with Pseudocode | |
| 3) Conclusion ---------- | 14 |
| a) Problems we faced during coding | |
| b) Limitations of the code | |
| c) Lesson Learned | |
| d) Project Success | |
| 4) Output Snapshots ---------- | 16 |
| 5) Source Code ---------- | 18 |

# "Implementation of Dynamic Table"

## Introduction:

- A **dynamic table** typically refers to a table, spreadsheet, or data structure in which the number of rows and columns can change or be modified dynamically, as opposed to a static table with fixed dimensions. This means that you can add, remove, or adjust rows and columns in response to changing data or user interactions.
- Dynamic tables are commonly used in software applications, databases, and web development to manage and display data in a flexible and adaptable manner. They are especially useful when dealing with datasets of varying sizes or when user-generated content needs to be accommodated.
- *Some key features of dynamic tables:*
  - **Dynamic Sizing:** Dynamic tables can resize themselves as elements are added or removed. This makes them suitable for situations where the number of elements is not known in advance or can change over time.
  - **Efficient Memory Usage:** Dynamic tables allocate memory as needed, which can help avoid wasting memory when dealing with varying data sizes. However, there is often some overhead associated with dynamic memory management.
  - **Random Access:** Similar to static tables, dynamic tables typically allow for random access to elements based on their indices.
  - **Insertions and Deletions:** Dynamic tables can efficiently handle insertions and deletions of elements, but the performance may vary depending on the data structure used.
  - **Automatic Memory Management:** In languages with garbage collection, dynamic tables can handle memory management automatically, reducing the risk of memory leaks or buffer overflows.
  - **Flexibility:** Dynamic tables are versatile and can be used in a wide range of applications, from simple lists to more complex data structures like stacks, queues, and hash tables.

# Methodology:

## Approach:

We used Array (Dynamic) to create Dynamic Table along with Dynamic memory Allocation and used variables and pointers to connect between functions and variables.

## Process:

The program works by calling different functions displayed in a menu. It will create a Dynamic Table, Insert Row & Column; Delete Row & Column by resizing the Table each time.

**Textual representation of the code for implementation of Dynamic Table:**

```
Start
|
|--- Menu (Display Options: 1. Create Dynamic Table, 2. Insert,
          3.Delete, 4. Display Table, 5. Create New Table, 6. Exit)
|     |
|     |--- Option 1: Create Dynamic Table
|     |     |
|     |     |--- Check if table already exists
|     |     |     |
|     |     |     |--- Yes: Display "Table already created" message
|     |     |     |
|     |     |     |--- No: Call create() function to create a dynamic
                        table
|     |
|     |--- Option 2: Insert (Row or Column)
|     |     |
|     |     |--- Check if a table exists
|     |     |     |
|     |     |     |--- Yes: Ask for Row or Column insertion
|     |     |     |     |
|     |     |     |     |--- Option 1: Insert Row
|     |     |     |     |     |
|     |     |     |     |     |--- Ask for the row number
|     |     |     |     |     |--- Call insertRow() function
|     |     |     |     |
|     |     |     |     |--- Option 2: Insert Column
|     |     |     |     |     |
|     |     |     |     |     |--- Ask for the column number
|     |     |     |     |     |--- Call insertCol() function
|     |     |     |     |
|     |     |     |     |--- Invalid Choice: Display "Invalid Choice"
                        message
|     |     |
|     |     |--- No: Display "Table Not created Yet" message
|     |
```

```
|     |--- Option 3: Delete (Row or Column)
|     |     |
|     |     |--- Check if a table exists
|     |     |     |
|     |     |     |--- Yes: Ask for Row or Column deletion
|     |     |     |     |
|     |     |     |     |--- Option 1: Delete Row
|     |     |     |     |     |
|     |     |     |     |     |--- Ask for the row number
|     |     |     |     |     |--- Call delRow() function
|     |     |     |     |
|     |     |     |     |--- Option 2: Delete Column
|     |     |     |     |     |
|     |     |     |     |     |--- Ask for the column number
|     |     |     |     |     |--- Call delColumn() function
|     |     |     |     |
|     |     |     |     |--- Invalid Choice: Display "Invalid Choice"
                              message
|     |     |
|     |     |--- No: Display "Table Not created Yet" message
|     |
|     |--- Option 4: Display Table
|     |     |
|     |     |--- Check if a table exists
|     |     |     |
|     |     |     |--- Yes: Call display() function to display the table
|     |     |
|     |     |--- No: Display "Table Not created Yet" message
|     |
|     |--- Option 5: Create New Table (Deleting the old one)
|     |     |
|     |     |--- Check if a table exists
|     |     |     |
|     |     |     |--- Yes: Display "Creating New Dynamic Table. Deleting
                              the previous one." message
|     |     |     |     |
|     |     |     |     |--- Free the existing table using freeDT()function
|     |     |     |     |--- Call create() function to create a new dynamic
                              table
|     |     |
|     |     |--- No: Call create() function to create a new dynamic table
|     |
|     |--- Option 6: Exit
|     |     |
|     |     |--- Check if a table exists
|     |     |     |
|     |     |     |--- Yes: Free the existing table using freeDT() function
|     |     |
|     |     |--- Display "Exiting the program." message
|     |
|     |--- Invalid Choice: Display "Invalid Choice" message
|
|--- Repeat the Menu until Option 6 (Exit) is chosen
|--- after exiting menu will delete any existing dynamic table to free
     space
|
End
```

## Detail Explanation of each Function with Pseudocode:

Every function of this program will check if a table exists or not. The insertion and Deletion operations can be done only when a table exists & if table is not created yet, it will ask user to create a table first.

### Main():

In main function, it will display a menu which will show all the option. There is a **Switch Statement** where each case holds a **different function**, will be called with **user input** and will continue to ask until user ask to terminate the program.
There's a case where user can create a completely new table by deleting the existed table as All operations can be done only on the existed table.

### Structures:

The code has 3 structures –

1. `dynamic_table` – which consist number of row & column and has 3D Array for cells and each Column information
2. `TableColumn` – consist each column name and it's type
3. `TableCell` – consist each cell data (**a void pointer**) and it's type. Here, **void pointer** is used so that any type of Data Can be stored in cells.

### create():

This function creates a dynamic table from scratch. Below is it's **pseudo code with explanations on each logic:**

```
Function create() -> dynamic_table:
    // Create a new dynamic_table structure
    table = Allocate memory for dynamic_table

    // Prompt the user to input the number of columns
    Print "Enter # of columns for a Dynamic Table: "
    Read table.no_columns

    // Check if the number of columns is non-positive
    If table.no_columns <= 0 Then
        Print "No columns specified, exiting the program"
        Exit with error code 1
```

```
    End If

    // Allocate memory for the column structure
    table.column = Allocate memory for TableColumn[table.no_columns]

    // Loop to input column names and data types
    For i = 0 To table.no_columns - 1 Do
        Print "Enter Title for Column " + (i + 1) + " : "
        Read table.column[i].name
        Print "Select Data Type: 1. Integer\t2. Decimal\t3. String"
        Print "Your Choice: "
        Read table.column[i].type
    End For

    // Prompt the user to input the number of rows
    Print "Enter # of rows for a Dynamic Table: "
    Read table.no_row

    // Allocate memory for the data array (rows x columns)
    table.data = Allocate memory for TableCell*[table.no_row]

    // Loop to initialize the data array with nullptrs
    For i = 0 To table.no_row - 1 Do
        table.data[i] = Allocate memory for
TableCell*[table.no_columns]
        For j = 0 To table.no_columns - 1 Do
            table.data[i][j] = nullptr
        End For
    End For
    Print "Table Created Successfully."
    // Loop to input data for each cell in the table
    For i = 0 To table.no_row - 1 Do
        For j = 0 To table.no_columns - 1 Do
            type = table.column[j].type
            data = NULL

            Switch type:
                Case INT:
                    Allocate memory for data as int
                    Print "Enter Integer Value for " + (i + 1) + "
and " + (j + 1) + " : "
                    Read data as int
                Case DOUBLE:
                    Allocate memory for data as double
                    Print "Enter Decimal Value for " + (i + 1) + "
and " + (j + 1) + " : "
                    Read data as double
                Case STRING:
```

```
                    Allocate memory for data as char[STRING_LENGTH]
                    Print "Enter String Value for " + (i + 1) + "
and " + (j + 1) + " : "
                    Ignore previous newline character
                    Read a line of input into data
            End Switch

            // Create a new TableCell and store data and type
            table.data[i][j] = Allocate memory for TableCell
            table.data[i][j].type = type
            table.data[i][j].data = data
        End For
    End For

    // Return the dynamically created table
    Return table
End Function
```

- ## freeDT(table):

  This function will delete the pre-existed Dynamic Table. As Insert Row & Column can overwrite a cell, thus new table can not be made unless we delete the existing one. Thus this function delete the existing Table by it's cells one by one by going through column & row.

- ## insertRow(table, row):

  Below is it's **pseudo code with explanations on each logic:**

```
function insertRow(table: dynamicTable, row: integer)
    // Check if the specified row position is invalid
    if row < 0 or row > table.noRows then
         return
    end if
// Increment the number of rows in the table
table.noRows++
 // Reallocate memory for the table's data
    table.data <- realloc(table.data, table.noRows *
sizeof(TableCell***))
    if table.data is null then
        exit(1)
    end if

 // Shift existing rows down to make space for the new row
    for i <- (table.noRows - 1) down to (row + 1) do
        table.data[i] <- table.data[i - 1]
```

```
    end for

// Allocate memory for the new row
    table.data[row] <- new TableCell*[table.noColumns]

// Initialize the new row with nullptr values for each cell
    for j <- 0 to table.noColumns - 1 do
        table.data[row][j] <- null
    end for

// Populate the new row with data
           for j <- 0 to table.noColumns - 1 do
 // Determine the data type for the current column
                  type <- table.column[j].type

 // Initialize a variable to store the cell's data
 // Use a switch statement to handle data input based on the data
type
                switch type do
                      /*As per in create()*/
                end switch

   // Create a new TableCell object and assign the data
                 table.data[row][j] <- new TableCell
                table.data[row][j].type <- type
                table.data[row][j].data <- data
           end for
    end for

    // Notify the user that the new row has been inserted
    end function
```

- ## insertCol(table, column-th):

  This function was the most error one, had to redo it again and again. As it was overwriting other columns data or not showing added column or terminating the whole program. We had to redo the code many times; we took help from different sites and did a more research on this function. Below is its **pseudo code with explanations on each logic:**

```
Function insertCol(table: dynamic_table, col: int):
    // Check if the column position is valid
    If col < 0 OR col > table.no_columns Then
        Print "Invalid Column Position to Insert"
        Return
    End If
```

```
            // Increment the number of columns
            table.no_columns++

            // Create a new array for the column structure
            newColumn = Allocate memory for
           TableColumn[table.no_columns]

// Copy the old columns to the new array, up to the insertion point;
/*so that inserted column does not overwrite existed columns*/
            For j = 0 To col - 1 Do
                Copy table.column[j].name to newColumn[j].name
                Copy table.column[j].type to newColumn[j].type
            End For

 // Get the name and type for the new column from user input
            Print "Enter the name for the new column: "
            Read newColumn[col].name
            Print "Select Data Type: 1. Integer\t2. Decimal\t3.
        String"
            Print "Your Choice: "
            Read newColumn[col].type

// Copy the old columns to the new array, after the insertion point
            For j = col To table.no_columns - 2 Do
                Copy table.column[j].name to newColumn[j + 1].name
                Copy table.column[j].type to newColumn[j + 1].type
            End For

          // Free the old column array
            Deallocate memory for table.column

            // Update the table to use the new column array
            table.column = newColumn

        // Reallocate memory for data in each row for the new column
            For i = 0 To table.no_row - 1 Do
                Allocate memory for table.data[i] with size
        table.no_columns * sizeof(TableCell*)

// Shift the data in existing cells to the right for the new column
                For j = table.no_columns - 2 Down To col Do
                    table.data[i][j + 1] = table.data[i][j]
                End For

           // Initialize only the new cell in the inserted column
                type = table.column[col].type
                data = NULL
```

```
                Switch type:
                 /*As per create()function to create Dynamic table*/
                End Switch

                Allocate memory for table.data[i][col] as TableCell
                table.data[i][col].type = type
                table.data[i][col].data = data
            End For

            Print "New Column Insertion Complete"
End Function
```

## delRow(table, n-th Row):

This function works by deleting an entire row given by user and resize the dynamic table accordingly. Below is it's **pseudo code with explanations on each logic:**

```
Function delRow(table: dynamic_table, row: int):
    // Check if the row position is valid
    If row < 0 OR row >= table.no_row Then
        Print "Invalid Row Position to Delete"
        Return
    End If

    // Loop through each column in the specified row
    For j = 0 To table.no_columns - 1 Do
        // Check if a cell exists in the current position
        If table.data[row][j] is not NULL Then
            // Free the memory occupied by cell data
            Deallocate memory for table.data[row][j].data
            // Free the memory occupied by the cell itself
            Deallocate memory for table.data[row][j]
        End If
    End For

    // Free the memory occupied by the row's data array
    // Shift rows above the deleted row down by one position
    For i = row To table.no_row - 2 Do
        table.data[i] = table.data[i + 1]
    End For

    // Decrement the total number of rows in the table
    table.no_row--
```

```
 // Print a message indicating that row deletion is complete
End Function
```

## delCol(table, n-th column):

This function works by deleting an entire column given by user and resize the dynamic table accordingly. Below is it's **pseudo code with explanations on each logic:**

```
Function delColumn(table: dynamic_table, col: int):
    // Check if the column position is valid
    If col < 0 OR col >= table.no_columns Then
        Print "Invalid Column Position to Delete"
        Return
    End If

    // Loop through each row in the table
    For i = 0 To table.no_row - 1 Do
    // Check if a cell exists in the current column position
        If table.data[i][col] is not NULL Then
            // Free the memory occupied by cell data
            Deallocate memory for table.data[i][col].data
            // Free the memory occupied by the cell itself
            Deallocate memory for table.data[i][col]
        End If

// Shift cells in columns to the left starting from the
              deleted column
        For j = col To table.no_columns - 2 Do
            table.data[i][j] = table.data[i][j + 1]
        End For
    End For

// Shift column names and types to the left starting from the
deleted column
    For j = col To table.no_columns - 2 Do
        Copy table.column[j + 1].name to
table.column[j].name
        Copy table.column[j + 1].type to
table.column[j].type
    End For

    // Decrement the total number of columns in the table
    table.no_columns--

// Print a message indicating that column deletion is complete
    End Function
```

## display(table):

This will display the whole Dynamic table in a table manner and show all the inserted information. Below is it's **pseudo code with explanations on each logic:**

```
Function display(table: dynamic_table):
    Print a newline

    // Display column headers
    For j = 0 To table.no_columns - 1 Do
        Print "\t" + table.column[j].name + "\t"
    End For
    Print a newline

    // Loop through each row and column to display the data
    For i = 0 To table.no_row - 1 Do
        For j = 0 To table.no_columns - 1 Do
           // Check if a cell exists in the current position
            If table.data[i][j] is not NULL Then
                // Get the cell and its type
                cell = table.data[i][j]
                cell_type = cell.type

                // Display the cell's data based on its type
                Switch cell_type:
                    Case INT:
                        Print "\t" +
   String(*((int*)cell.data)) + "\t"
                    Case DOUBLE:
                        Print "\t" +
String(*((double*)cell.data)) + "\t"
                    Case STRING:
                        Print "\t" + (char*)cell.data + "\t"
                End Switch
            Else
                // Display a placeholder for empty cells
                Print "\t--\t"
            End If
        End For
        Print a newline
    End For
End Function
```

# Conclusion:

## Problems we faced during coding:

- There are some warnings in our code which occurs because of use of **void pointer.** As to use void pointer in any data type, need to type cast it every time, it was showing warnings. But this pointer was used solely to have any type of data by overcoming the limitations of data types.
- There was a 3 pointer variable which was used first by us. It was little bit tricky for us to handle. At first we used 2 pointer but later was having many errors which led to us believe we should go for 3-pointer variable for the 3D array where cell information was stored.
- During column insertion, column name was printing but while printing information it was terminating the whole program, after some changes it was overwriting the previous column, but later after some help we managed to solve this problem

## Limitations of the code:

- Our program only takes 3 data type values

```
Print "Select Data Type: 1. Integer\t2. Decimal\t3. String"
```

  But additional data types like – Boolean could have been added. We tried to add Boolean data type, was creating many problems for bool data type, mainly when we were trying to typecast it.

```
case BOOL:
     data = malloc(sizeof(bool));;
     if (!data) {
         cout<<"Memory allocation failed"<<endl;
         exit(1);
     }
    cout<<"Enter Boolean Value for "<<i + 1<<" and "<<j + 1<<" : ";
    scanf("%s", (bool*)data);
    break;
```

  Because of the type caste issue,we had to remove this case.
- If any data type is typed in place of another then the program will **turn into a Infinite loop**, which a big limitation of the whole project.

- Another feature we tried adding was inspired from Hashmap, where deleted cell would show Deleted sign. We attempted to this but did not get desire output.

```
struct TableCell {
    int type;
    void* data;
    bool del;/*if true then had shown Deleted on that particular
            cell, but was not kept as we later aimed to delete an
            entire row or an entire column */
};
```

## Lesson Learned:

1. We had a lot practice as we were trying to solve the problems or errors to fix the code.
2. We had a great practice on using Pointers as we nedded to keep track of all the works using pointer.
3. We needed lot of information on our problematic part of code, thus we browse a lot of problem which enabled us to know more about the usages of Dynamic Table and it's implementation

## Project Success:

- Our submitted project compiles Successfully without any error.
- If while giving input, user gives something that is undefined it will show message of re-enter their wish, sometimes it takes it back to menu for another glimpse of the options, such as

```
*********************************************
                MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 3
Delete Row or Column? 1. Row    2. Column       3. A cell
 Choice: 5
Invalid Choice!!!! Taking you to Menu.
*********************************************
                MENU
```

It shows that user has given wrong input, so it takes the user back to menu.

Thus, it concludes our project report with pseudo codes and explanations.

Below is our source code and output:

## ✵ Output:

```
                    MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 1
Enter # of columns for a Dynamic Table: 3
Enter Title for Column 1 : Name
Select Data Type: 1. Integer    2. Decimal      3. String
Your Choice: 3
Enter Title for Column 2 : Age
Select Data Type: 1. Integer    2. Decimal      3. String
Your Choice: 1
Enter Title for Column 3 : Income
Select Data Type: 1. Integer    2. Decimal      3. String
Your Choice: 2
Enter # of rows for a Dynamic Table: 2
Table Created Successfully.
Enter String Value for 1 and 1 : Rifa
Enter Integer Value for 1 and 2 : 23
Enter Decimal Value for 1 and 3 : 435.23
Enter String Value for 2 and 1 : Tasnova
Enter Integer Value for 2 and 2 : 56
Enter Decimal Value for 2 and 3 : 23543.356
*********************************************
```

Dynamic Table Create

```
                    MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 4

        Name            Age             Income
        Rifa            23              435.23
        Tasnova         56              23543.4
*********************************************
```

Dynamic Table Display

```
6. Exit
Enter your choice: 2
Insert Row or Column? 1. Row    2. Column
 Choice: 1
Enter n-th row to Insert: 2
Enter String Value for 2 and 1 : Adidas
Enter Integer Value for 2 and 2 : 45
Enter Decimal Value for 2 and 3 : 23.34
New Row Insertion Complete.
*********************************************
                MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 4

        Name            Age             Income
        Rifa            23              435.23
        Adidas          45              23.34
        Tasnova         56              23543.4
*********************************************
```

```
Enter your choice: 2
Insert Row or Column? 1. Row    2. Column
 Choice: 2
Enter n-th Column to Insert: 2
Enter the name for the new column: Gendar
Select Data Type: 1. Integer    2. Decimal      3. String
Your Choice: 3
Enter String Value for 1 and 2 : Female
Enter String Value for 2 and 2 : Male
New Column Insertion Complete.
*********************************************
                MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 4

        Name            Gendar          Age             Income
        Rifa            Female          23              435.23
        Tasnova         ale             56              23543.4
*********************************************
```

```
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 3
Delete Row or Column? 1. Row    2. Column
 Choice: 1
Enter n-th row to Delete: 2
Row Deletion Done.
*********************************************
                MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 4

        Name            Age             Income
        Rifa            23              435.23
        Tasnova         56              23543.4
*********************************************
```

```
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 3
Delete Row or Column? 1. Row    2. Column
 Choice: 2
Enter n-th Column to Delete: 2
Column Deletion Done.
*********************************************
                MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 4

        Name            Age             Income
        Rifa            23              435.23
        Tasnova         56              23543.4
*********************************************
```

```
                MENU
1. Create Dynamic Table
2. Insert
3. Delete
4. Display table
5. Create New Table(Deleting the old one)
6. Exit
Enter your choice: 6
Exiting the program.
PS C:\Users\beaut\Documents\C++\Project>
```

Insertion (Row& Column)
Deletion(Row& Column)
Exit Program

# ✵ Source Code:

```
#include <iostream>
#include <cstdlib>
#include <cstring>

using namespace std;

#define STRING_LENGTH 180
/*Data Types, use this macros to define which data types
we using IN PATICULAR CELL; easy to code*/
#define INT 1
#define DOUBLE 2
#define STRING 3

struct TableCell {
    int type;
    void* data;//can point to any type data, no specification
};

struct TableColumn {
    char name[STRING_LENGTH];
    int type;
};

struct dynamic_table {
    int no_row, no_columns;
    TableCell*** data;//3d Array;
/*Cell datatype of a paticular individual cell*/
    TableColumn* column;
};
    /*Dynamic table create*/
dynamic_table* create() {
    dynamic_table* table = new dynamic_table;

    cout << "Enter # of columns for a Dynamic Table: ";
    cin >> table->no_columns;

    if (table->no_columns <= 0) {
        cout << "No columns specified, exiting the program" << endl;
        exit(1);
    }
    /*assign memory for specified columns*/
    table->column = new TableColumn[table->no_columns];
     /*Column Title assign*/
    for (int i = 0; i < table->no_columns; i++) {
        cout << "Enter Title for Column " << i + 1 << " : ";
        cin >> table->column[i].name;
```

```
        cout << "Select Data Type: 1. Integer\t2. Decimal\t3. String\n";
        cout << "Your Choice: ";
        cin >> table->column[i].type;
    }

    cout << "Enter # of rows for a Dynamic Table: ";
    cin >> table->no_row;
    /*Cell create/ Create Data Array*/
    table->data = new TableCell**[table->no_row];

    for (int i = 0; i < table->no_row; i++) {
        table->data[i] = new TableCell*[table->no_columns];
        for (int j = 0; j < table->no_columns; j++)
            table->data[i][j] = nullptr;//initialize all to NULL
    }

    cout << "Table Created Successfully." << endl;
        /*User Input Values*/
    for (int i = 0; i < table->no_row; i++) {
        for (int j = 0; j < table->no_columns; j++) {
            int type = table->column[j].type;
            void* data = nullptr;

            switch (type) {
                case INT:
                    data = new int;
                    cout << "Enter Integer Value for " << i + 1 << " and " <<
                         j + 1 << " : ";
                    cin >> *((int*)data);
                    break;
                case DOUBLE:
                    data = new double;
                    cout << "Enter Decimal Value for " << i + 1 << " and " <<
                         j + 1 << " : ";
                    cin >> *((double*)data);
                    break;
                case STRING:
                    data = new char[STRING_LENGTH];
                    cout << "Enter String Value for " << i + 1 << " and " << j
                         + 1 << " : ";
                    cin.ignore();  // Ignore previous newline character
                    cin.getline((char*)data, STRING_LENGTH);
                    break;
            }

            table->data[i][j] = new TableCell;
            table->data[i][j]->type = type;
            table->data[i][j]->data = data;
```

```
        }
    }

    return table;
}
/*to free the whole Dynamic Table*/
void freeDT(dynamic_table* table) {
    /*free everything in reverse as created to be safe*/
    for (int i = 0; i < table->no_row; i++) {
        for (int j = 0; j < table->no_columns; j++) {
            if (table->data[i][j]) {
                delete table->data[i][j]->data;
                delete table->data[i][j];
            }
        }
        delete[] table->data[i];
    }
    delete[] table->data;
    delete[] table->column;
    delete table;
}

void insertRow(dynamic_table* table, int row) {
    if (row < 0 || row > table->no_row) {
        cout << "Invalid Row Position to Insert" << endl;
        return;
    }

    // Increment the number of rows
    table->no_row++;

    // Reallocate memory for data
    table->data = (TableCell***)realloc(table->data, table->no_row *
sizeof(TableCell***));
    if (!table->data) {
        cout << "Memory Allocation for Cell Failed!!!!!" << endl;
        exit(1);
    }

   /*to avoid overwrite the existing data in the table*/
    // Shift existing rows down
    for (int i = table->no_row - 1; i > row; i--) {
        table->data[i] = table->data[i - 1];
    }

    // Allocate memory for the new row
    table->data[row] = new TableCell*[table->no_columns];
    for (int j = 0; j < table->no_columns; j++) {
```

```
        table->data[row][j] = nullptr;
    }

    // Populate the new row
    for (int j = 0; j < table->no_columns; j++) {
        int type = table->column[j].type;
        void* data = nullptr;

        switch (type) {
            case INT:
                data = new int;
                cout << "Enter Integer Value for " << row + 1 << " and " << j
                         + 1 << " : ";
                cin >> *((int*)data);
                break;
            case DOUBLE:
                data = new double;
                cout << "Enter Decimal Value for " << row + 1 << " and " << j
                         + 1 << " : ";
                cin >> *((double*)data);
                break;
            case STRING:
                data = new char[STRING_LENGTH];
                cout << "Enter String Value for " << row + 1 << " and " << j +
                         1 << " : ";
                cin.ignore();  // Ignore previous newline character
                cin.getline((char*)data, STRING_LENGTH);
                break;
        }

        table->data[row][j] = new TableCell;
        table->data[row][j]->type = type;
        table->data[row][j]->data = data;
    }

    cout << "New Row Insertion Complete." << endl;
}

void insertCol(dynamic_table* table, int col) {
    if (col < 0 || col > table->no_columns) {
        cout << "Invalid Column Position to Insert" << endl;
        return;
    }

    // Increment the number of columns
    table->no_columns++;

    // Create a new array for the column structure
```

```
    TableColumn* newColumn = new TableColumn[table->no_columns];

    // Copy the old columns to the new array, up to the insertion point
    for (int j = 0; j < col; j++) {
        strcpy(newColumn[j].name, table->column[j].name);
        newColumn[j].type = table->column[j].type;
    }

    // Get the name and type for the new column
    cout << "Enter the name for the new column: ";
    cin >> newColumn[col].name;
    cout << "Select Data Type: 1. Integer\t2. Decimal\t3. String\n";
    cout << "Your Choice: ";
    cin >> newColumn[col].type;

    // Copy the old columns to the new array, after the insertion point
    for (int j = col; j < table->no_columns - 1; j++) {
        strcpy(newColumn[j + 1].name, table->column[j].name);
        newColumn[j + 1].type = table->column[j].type;
    }

    // Free the old column array
    delete[] table->column;

    // Update the table to use the new column array
    table->column = newColumn;

    // Reallocate memory for data in each row for the new column
    for (int i = 0; i < table->no_row; i++) {
        table->data[i] = (TableCell**)realloc(table->data[i], table-
                    >no_columns * sizeof(TableCell*));

        // Shift the data in existing cells to the right for the new column
        for (int j = table->no_columns - 2; j >= col; j--) {
            table->data[i][j + 1] = table->data[i][j];
        }

        // Initialize only the new cell in the inserted column
        int type = table->column[col].type;
        void* data = nullptr;

        switch (type) {
            case INT:
                data = new int;
                cout << "Enter Integer Value for " << i + 1 << " and " << col
                         + 1 << " : ";
                cin >> *((int*)data);
                break;
```

```
            case DOUBLE:
                data = new double;
                cout << "Enter Decimal Value for " << i + 1 << " and " << col
                   + 1 << " : ";
                cin >> *((double*)data);
                break;
            case STRING:
                data = new char[STRING_LENGTH];
                cout << "Enter String Value for " << i + 1 << " and " << col +
                   1 << " : ";
                cin.ignore();  // Ignore previous newline character
                cin.getline((char*)data, STRING_LENGTH);
                break;
        }

        table->data[i][col] = new TableCell;
        table->data[i][col]->type = type;
        table->data[i][col]->data = data;
    }

    cout << "New Column Insertion Complete." << endl;
}


void delRow(dynamic_table* table, int row) {
    if (row < 0 || row >= table->no_row) {
        cout << "Invalid Row Position to Delete" << endl;
        return;
    }
    /*free memory for each cell*/
    for (int j = 0; j < table->no_columns; j++) {
        if (table->data[row][j]) {
            delete table->data[row][j]->data;
            delete table->data[row][j];
        }
    }

    delete[] table->data[row];
    /*Shifting row upwards*/
    for (int i = row; i < table->no_row - 1; i++) {
        table->data[i] = table->data[i + 1];
    }

    table->no_row--;//decrement number of rows
    cout << "Row Deletion Done." << endl;
}

void delColumn(dynamic_table* table, int col) {
    if (col < 0 || col >= table->no_columns) {
```

```
        cout << "Invalid Column Position to Delete" << endl;
        return;
    }
        /*free memory for each cell*/
    for (int i = 0; i < table->no_row; i++) {
        if (table->data[i][col]) {
            delete table->data[i][col]->data;
            delete table->data[i][col];
        }

        for (int j = col; j < table->no_columns - 1; j++) {
            table->data[i][j] = table->data[i][j + 1];
        }
    }
        /*Column info Update*/
    for (int j = col; j < table->no_columns - 1; j++) {
        strcpy(table->column[j].name, table->column[j + 1].name);
        table->column[j].type = table->column[j + 1].type;
    }

    table->no_columns--;//decrement number of Column
    cout << "Column Deletion Done." << endl;
}

void display(dynamic_table* table) {
    cout<<endl;
    for (int j = 0; j < table->no_columns; j++) {
        cout <<"\t"<<table->column[j].name << "\t";
    }
    cout << endl;

    for (int i = 0; i < table->no_row; i++) {
        for (int j = 0; j < table->no_columns; j++) {
            if (table->data[i][j]) {
                TableCell* cell = table->data[i][j];
                switch (cell->type) {
                    case INT:
                        cout <<"\t"<< *((int*)cell->data) << "\t";
                        break;
                    case DOUBLE:
                        cout <<"\t"<< *((double*)cell->data) << "\t";
                        break;
                    case STRING:
                        cout <<"\t"<< (char*)cell->data << "\t";
                        break;
                }
            } else {
                cout <<"\t"<< "--\t";
```

```
            }
        }
        cout << endl;
    }
}
int main() {
    dynamic_table* table = nullptr;
    int menu;

    do {
        cout << "*********************************************" << endl;
        cout << "\t\tMENU\n"
             << "1. Create Dynamic Table\n2. Insert\n"
             << "3. Delete\n4. Display table\n"
             << "5. Create New Table(Deleting the old one)\n"
             << "6. Exit\nEnter your choice: ";
        cin >> menu;

        switch (menu) {
            case 1:
                if (table) {
                    cout << "Table already created. Please choose another
                         option." << endl;
                } else {
                    table = create();
                }
                break;
            case 2:
                if (!table) {
                    cout << "Table Not created Yet. Please create a table
                   first (Option 1)." << endl;
                } else {
                    int choice;
                    cout << "Insert Row or Column? 1. Row\t2. Column" << endl
                         << " Choice: ";
                    cin >> choice;
                    if (choice == 1) {
                        int row;
                        cout << "Enter n-th row to Insert: ";
                        cin >> row;
                        insertRow(table, row-1);
                    } else if (choice == 2) {
                        int col;
                        cout << "Enter n-th Column to Insert: ";
                        cin >> col;
                        insertCol(table, col-1);
                    } else {
```

```
                        cout << "Invalid Choice!!!! Taking you to Menu." <<
                         endl;
                    }
                }
                break;
            case 3:
                if (!table) {
                    cout << "Table Not created Yet. Please create a table
                         first (Option 1)." << endl;
                } else {
                    int choice;
                    cout << "Delete Row or Column? 1. Row\t2. Column\t" <<
                         endl << " Choice: ";
                    cin >> choice;
                    if (choice == 1) {
                        int row;
                        cout << "Enter n-th row to Delete: ";
                        cin >> row;
                        delRow(table, row-1);
                    } else if (choice == 2) {
                        int col;
                        cout << "Enter n-th Column to Delete: ";
                        cin >> col;
                        delColumn(table, col-1);
                    }else {
                        cout << "Invalid Choice!!!! Taking you to Menu." <<
                         endl;
                    }
                }
                break;
            case 4:
                if (!table) {
                    cout << "Table Not created Yet. Please create a table
                       first (Option 1)." << endl;
                } else {
                    display(table);
                }
                break;
            case 5:
                if (table) {
                    cout << "Creating New Dynamic Table. Deleting the previous
                         one." << endl;
                    freeDT(table);
                    table = nullptr;
                }
/*If table exists already then dlt the 1st one & create a new one*/
                table = create();
                break;
```

```
            case 6:
                if (table) {
                    freeDT(table);
                }
                cout << "Exiting the program." << endl;
                break;
            default:
                cout << "Invalid choice. Please try again.\n" << endl;
        }
    } while (menu != 6);
/*Before existing the programm dlt the table*/
    freeDT(table);
    return 0;
}
```